

Especificación inicial — Herramienta CLI para análisis y procesamiento de BLAST

Objetivo general

Desarrollar una herramienta de línea de comandos (CLI) que automatice dos flujos de trabajo relacionados con BLAST: (1) recibir secuencias FASTA/multifasta, determinar el algoritmo BLAST adecuado, ejecutar la búsqueda en formato tabular y generar la tabla final; y (2) recibir un archivo BLAST ya generado y un archivo de accesiones/descriptores para producir directamente la misma tabla final. El resultado será un archivo CSV con todas las columnas requeridas por la tabla de referencia del proyecto.

1. Alcance

- La herramienta se ejecutará desde terminal/CLI en Ubuntu/WSL.
- Se contemplan cinco algoritmos: blastn, blastp, blastx, tblastn y tblastx.
- El backend contiene la lógica de procesamiento; el CLI será integrado por otro integrante.
- La salida final será un CSV con todas las columnas definidas por la tabla de referencia.
- El usuario podrá indicar el nombre de salida; si no lo hace, será results.csv.

2. Regla de decisión de BLAST

La selección depende de la combinación del tipo de query y del tipo de base de datos. Para tblastx, NCBI confirma que tanto la query como las secuencias nucleotídicas de la base de datos se traducen en los seis marcos de lectura y se comparan a nivel de proteína.

Query	Base de datos	Algoritmo	Descripción
Nucleótido	Nucleótido	blastn	Nucleótido contra nucleótido.
Proteína	Proteína	blastp	Proteína contra proteína.
Nucleótido	Proteína	blastx	Query nucleotídica traducida en seis marcos contra proteínas.
Proteína	Nucleótido	tblastn	Proteína contra traducciones de seis marcos de una base nucleotídica.
Nucleótido	Nucleótido	tblastx	Query y base nucleotídica traducidas en seis marcos; comparación a nivel proteico.

Importante: blastn y tblastx pueden usar una base nucleotídica, pero no realizan la misma comparación. blastn compara directamente nucleótidos; tblastx traduce ambos lados y compara las traducciones.

3. Identificación de la secuencia

La entrada FASTA puede contener nucleótidos o aminoácidos. La regla de referencia del proyecto indica:

- Nucleótido: principalmente A, T, C, G (o U en ARN).
- Proteína: aminoácidos; pueden aparecer letras como L, P, F, E, Q, K, etc.

4. Camino 1 — FASTA/multifasta → BLAST → CSV

Entradas:

- Archivo FASTA/multifasta (.fa).
- Archivo de accesiones y descriptores.
- Base(s) de datos de búsqueda.
- Nombre opcional del archivo de salida.

Procesamiento:

1. Validar entradas.
2. Leer las secuencias FASTA.
3. Identificar el tipo de query.
4. Determinar el tipo de base de datos.
5. Seleccionar blastn, blastp, blastx, tblastn o tblastx.
6. Ejecutar BLAST+.
7. Obtener la salida en formato tabular.
8. Parsear los resultados.
9. Leer accesiones y descriptores.
10. Relacionar BLAST con los metadatos.
11. Construir la tabla final con todas las columnas requeridas.
12. Generar el CSV.

5. Camino 2 — BLAST existente → CSV

Entradas:

- Archivo BLAST tabular.
- Archivo de accesiones y descriptores.
- Nombre opcional del archivo de salida.

Procesamiento:

13. Validar entradas.
14. Parsear el archivo BLAST.
15. Leer accesiones y descriptores.
16. Relacionar resultados con los metadatos.
17. Construir la misma tabla final del Camino 1.
18. Generar el CSV.

6. Salida

Ambos caminos producen la misma estructura de salida. Todas las columnas de la tabla de referencia son obligatorias.

Nombre personalizado: el indicado por el usuario.

Nombre por defecto: results.csv.

7. Arquitectura propuesta del backend

- sequence/: lectura, validación y clasificación de FASTA.
- blast/: selección del algoritmo, construcción de comandos y ejecución de BLAST+.
- blast_parser/: lectura y normalización de resultados tabulares.
- metadata/: lectura de accesiones y descriptores.
- processing/: unión de BLAST con metadatos y construcción de la tabla final.
- output/: generación del CSV.
- pipeline/: coordinación de los dos caminos.

Flujo compartido:

Camino 1: FASTA → identificación → selección BLAST → ejecución → resultado tabular → procesamiento común → CSV.

Camino 2: BLAST tabular → procesamiento común → CSV.

8. Librerías y herramientas candidatas

- Python — backend.
- Biopython — lectura y manejo de FASTA/secuencias.
- pandas — transformación, unión y escritura de tablas/CSV.
- NCBI BLAST+ — blastn, blastp, blastx, tblastn y tblastx.
- subprocess — ejecución de BLAST+ desde Python.
- pathlib — manejo de rutas.

9. Interfaz backend ↔ CLI

Funciones candidatas:

- detect_sequence_type()
- detect_database_type()
- select_blast_program()
- run_blast()
- parse_blast_results()
- load_accessions()
- merge_blast_with_metadata()
- build_result_table()
- write_results_csv()

10. Requisitos funcionales iniciales

- RF-01: aceptar FASTA/multifasta en el Camino 1.
- RF-02: identificar query nucleotídica o proteica.
- RF-03: identificar el tipo de base de datos.
- RF-04: seleccionar entre los cinco algoritmos BLAST.
- RF-05: ejecutar BLAST+ y obtener salida tabular.
- RF-06: aceptar un BLAST tabular existente en el Camino 2.
- RF-07: aceptar accesiones y descriptores.
- RF-08: integrar BLAST y metadatos.
- RF-09: generar todas las columnas obligatorias.
- RF-10: permitir nombre personalizado.
- RF-11: usar results.csv por defecto.

11. Especificaciones

- Formato exacto del archivo de accesiones/descriptores: txt.
- Formato BLAST tabular aceptado en el Camino 2: txt, excel, csv.

12. Flujo general

Entrada → validación → selección del camino → identificación de tipos (si aplica) → selección/ejecución de BLAST (solo Camino 1) → parseo → integración de accesiones/descriptores → tabla final → CSV.

13. Verificación de tblastx

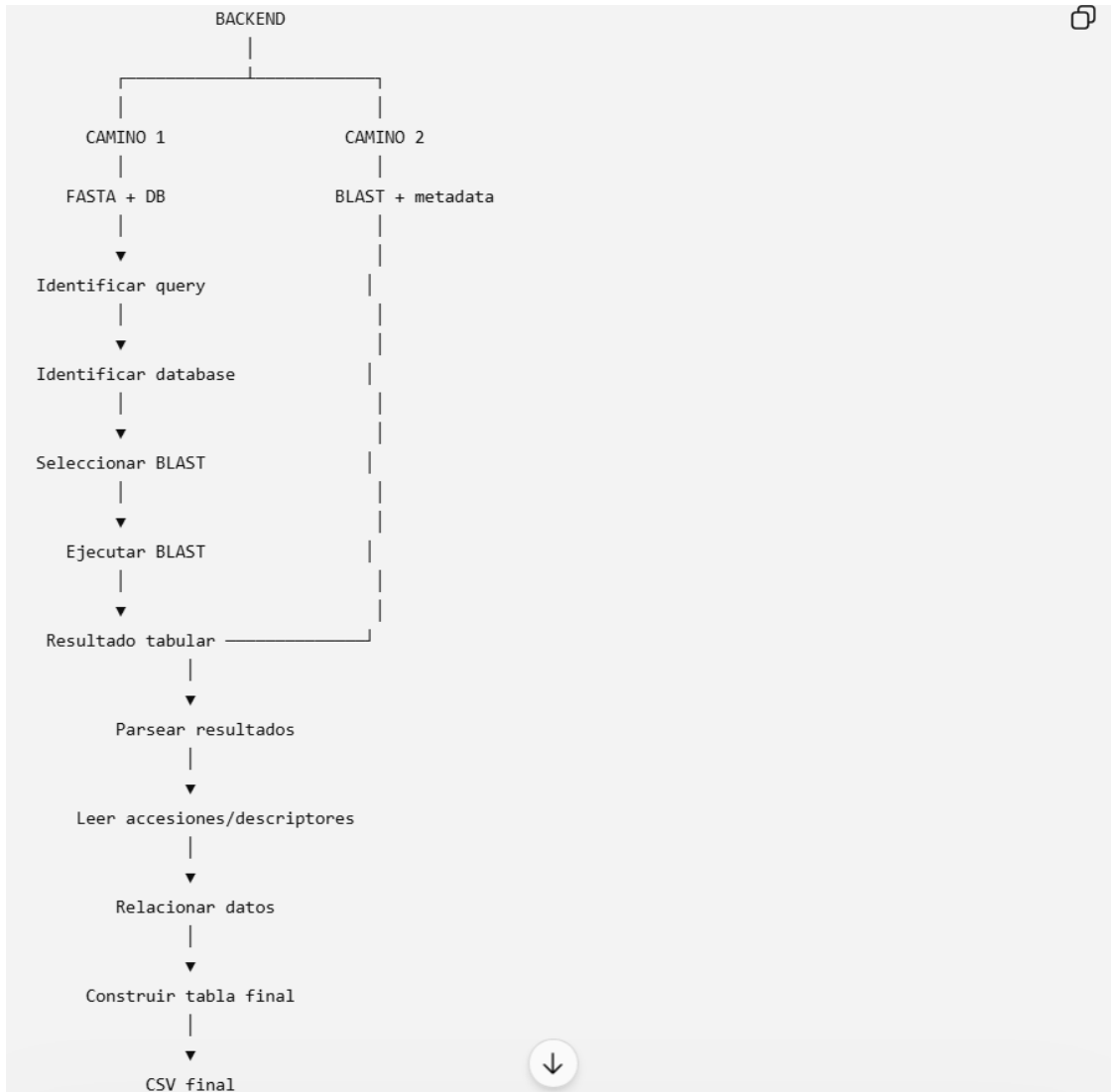
La regla propuesta es correcta. NCBI describe tblastx como una búsqueda donde una query nucleotídica y una base de datos nucleotídica se traducen en los seis marcos de lectura y se comparan las traducciones a nivel de proteína. En términos de marcos, se realizan comparaciones entre las seis traducciones de la query y las seis del sujeto.

14. Referencias técnicas

NCBI BLAST QuickStart — sección Translated BLAST.

NCBI BLAST+ — programas incluidos en el paquete BLAST+.

15. Flujo esperado



16. Resumen

La herramienta será una **CLI para automatizar dos flujos distintos**, pero ambos terminarán produciendo exactamente el mismo tipo de archivo:

Un CSV con todas las columnas requeridas de la tabla de resultados.

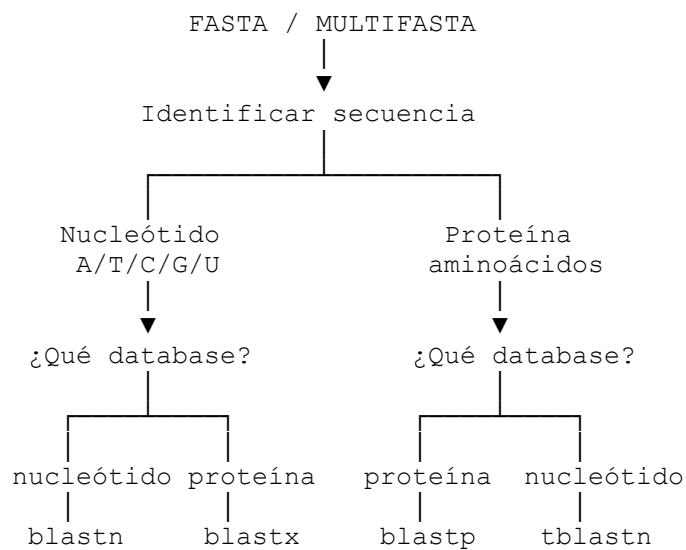
No habrá columnas "opcionales" en la salida: **todas las columnas de la tabla son necesarias.**

Camino 1 — FASTA → BLAST → CSV

El usuario proporciona:

1. Archivo FASTA/MULTIFASTA (.fa)
2. Archivo de accesiones + descriptores
3. Base(s) de datos donde buscar
4. Opcionalmente, nombre del archivo de salida

El flujo será:



Pero aquí hay una precisión importante:

La elección **no depende solamente de si el FASTA contiene nucleótidos o proteínas.**

Depende de la combinación:

TIPO DE QUERY
+
TIPO DE BASE DE DATOS
↓
TIPO DE BLAST

Después de elegir el BLAST

Una vez determinado, por ejemplo:

```
Query: proteína
Database: nucleótidos
      ↓
      tblastn
```

la herramienta ejecutará BLAST y **obligatoriamente obtendrá los resultados en formato tabular**.

Algo conceptualmente como:

```
FASTA
  ↓
tipo de secuencia
  ↓
tipo de BLAST
  ↓
BLAST+
  ↓
resultado tabular
  ↓
procesamiento
```

Después entra el archivo de accesiones/descriptores:

```
Resultado BLAST
      +
Accesiones + descriptores
      ↓
      procesamiento
      ↓
      tabla final
      ↓
      CSV
```

Nombre del archivo

También queda definido:

Si el usuario proporciona un nombre:

```
--output mi_resultado.csv
```

se utiliza ese nombre.

Si no proporciona ninguno:

results.csv

Por defecto.

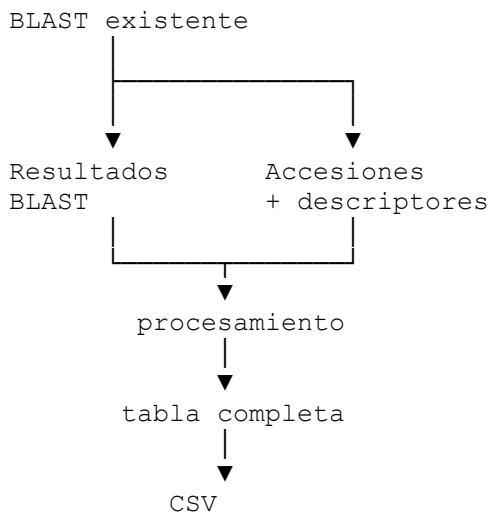
Camino 2 — BLAST existente → CSV

Este camino es diferente porque **NO se ejecuta BLAST**.

El usuario ya tiene:

Archivo BLAST
+
Archivo accesiones/descriptores

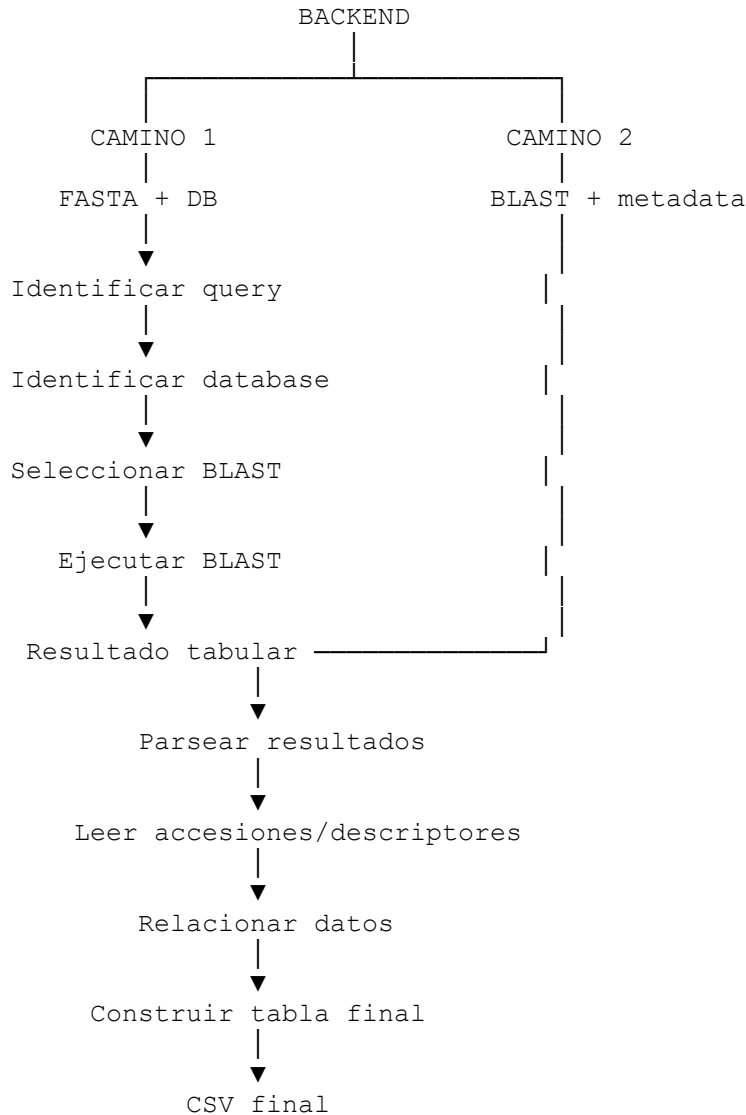
Entonces:



Por lo tanto, **el Camino 2 es esencialmente la parte de procesamiento de resultados del Camino 1**, pero sin la etapa de ejecución de BLAST.

Cómo veo ahora la arquitectura

Creo que podemos separar el backend en dos grandes partes:



Esto significa que podemos diseñar una función central de procesamiento que sea utilizada por **ambos caminos**.

el backend probablemente tendrá que encargarse de cosas como:

Entrada

- Leer FASTA/multifasta.
- Identificar si la query es nucleótido o proteína.
- Determinar el tipo de base de datos.
- Determinar cuál de los **5 BLAST posibles** utilizar.
- Ejecutar BLAST cuando corresponda.
- Leer archivos BLAST existentes.
- Leer el archivo de accesiones/descriptores.

Procesamiento

- Parsear los resultados tabulares.
- Relacionar los identificadores/accesiones.
- Incorporar los descriptores.
- Generar **todas las columnas requeridas**.
- Mantener el formato esperado de la tabla.

Salida

- Generar CSV.
- Permitir nombre personalizado.
- Usar `results.csv` si no se proporciona nombre.